八、部署大模型网络服务

我们基于LangChain框架可以快速构建一些基于大模型的应用，但是这些应用要发挥作用，还是需要部署一些类似于One-API，FastGPT之类的网络服务，才能对外提供更灵活的定制化服务。这一章节，就给大家介绍几种常用的基于大模型的网络服务开发的方式。

1、使用LangServer构建大模型网络服务

LangServer是一个用于构建和部署基于自然语言处理模型的应用程序的框架。他使得开发者可以轻松地将训练好的模型部署成为web服务，对外提供API结构，以便于其他应用程序进行调用。如果针对LangChain框架来说，那就是可以简单快速的把一个Chain暴露成网络服务，供其他应用访问。LangServer主打的就是一个功能完善，简单易上手。

1-安装langserver依赖

LangServer提供了统一的依赖，用来构建Web服务。

```
# 构建langserver服务端的依赖
#!pip install "langserve[server]"
# 构建langserver客户端的依赖
#!pip install "langserve[client]"
# 服务端+客户端整合依赖
!pip install "langserve[all]"
```

2-使用langchain构建一个chain

这里我们用langchain框架构建一个比较简单的chain用来演示langserve的使用方法。

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from config.load_key import load_key
from langchain_openai import ChatOpenAI

# 提示词模板
prompt_template = ChatPromptTemplate.from_messages([
    ("system","Translat the following from English into {language}"),
    ("user","{text}")
])
# 构建阿里云百炼大模型客户端
llm = ChatOpenAI(
    model="qwen-plus",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key=load_key("BAILIAN_API_KEY"),
)

# 结果解析器 StrOutputParser会AIMessage转换成为str，实际上就是获取AIMessage的content属性。
```

```
parser = StrOutputParser()
# 构建链
chain = prompt_template | llm | parser
# 直接调用链
print(chain.invoke({"text":"nice to meet you", "language":"Chinese"}))
```

3-构建langserver服务端

langserver中整合了一系列的工具和方法，使得构建一个服务端变得十分简单。

```
from fastapi import FastAPI
from langserve import add_routes
# 构建服务端对象
app = FastAPI(title="大模型语言翻译助手",version="v1.0",description="基于LangChain框架构建的大模型语言翻译助手")
# 添加路由信息
add_routes(
    app,
    chain,
    path="/langchainDemo")
# 启动服务
import uvicorn
uvicorn.run(app, host="0.0.0.0", port=8000)
```

注意，这个方法在jupyternotebook环境中是无法执行的，因为jupyter本身就需要启动一个web服务，你不可能在一个web服务中又启动一个新的Web服务。我们可以把这些代码整合到一个单独的python文件中。

```
!python ./langserver/myserver.py
```

启动完成后，就可以看到LangServe已经启动了多个访问端口：

```
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)

LANGSERVE: Playground for chain "/langchainDemo/" is live at:
LANGSERVE:  │
LANGSERVE:  └──> /langchainDemo/playground/
LANGSERVE:
LANGSERVE: See all available routes at /docs/
```

接下来，可以访问http://127.0.0.1:8000/docs查看服务端的文档。

localhost:8000/docs

AI工具 常用 Marscode 豆包功能介绍说明 AI 看视频: 是谁帮我... 2.0 AI 开发 - 飞书... 阿里

大模型语言翻译助手 v1.0 OAS 3.1

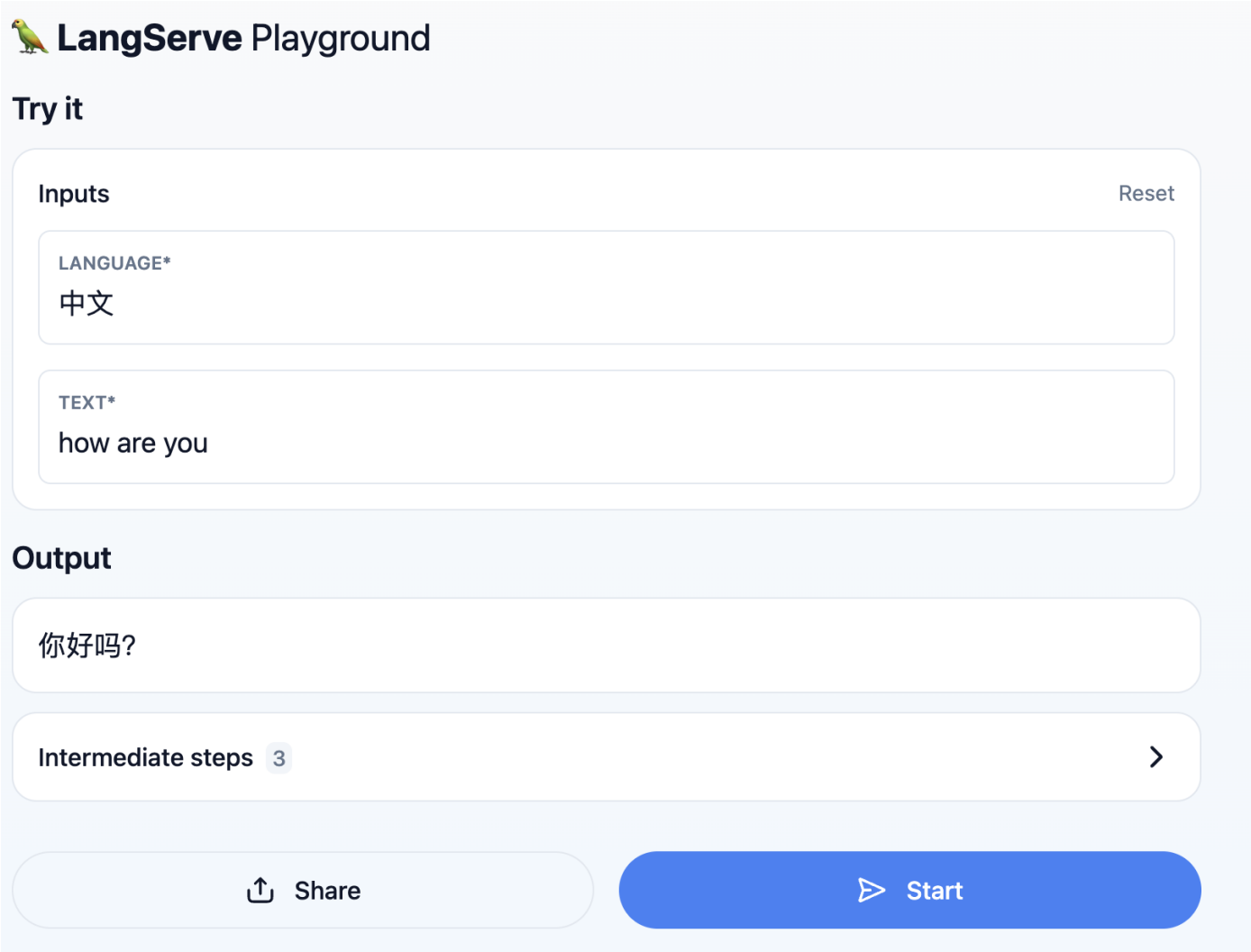
/openapi.json

基于LangChain框架构建的大模型语言翻译助手

langchainDemo

GET	/langchainDemo/input_schema	Langchaindemo Input Schema
GET	/langchainDemo/output_schema	Langchaindemo Output Schema
GET	/langchainDemo/config_schema	Langchaindemo Config Schema
POST	/langchainDemo/invoke	Langchaindemo Invoke
POST	/langchainDemo/batch	Langchaindemo Batch
POST	/langchainDemo/stream	Langchaindemo Stream
POST	/langchainDemo/stream_log	Langchaindemo Stream Log
POST	/langchainDemo/stream_events	Langchaindemo Stream Events

还可以访问http://localhost:8000/langchainDemo/playground/查看一个简单的交互界面。



当然，这个HTTP接口，也可以用POSTMAN这一类的HTTP工具来访问。

REST API basics: CRUD, test & variable / **Post data**

POST http://localhost:8000/langchainDemo/invoke

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
  "input": {
    "language": "Chinese",
    "text": "how are you "
  },
  "config": {},
  "kwargs": {}
}
```

Body Cookies Headers (4) Test Results (1/1) 200 OK 534 ms

{} JSON Preview Visualize

```
{
    "output": "你好吗? ",
    "metadata": {
        "run_id": "a745733c-2eee-43ed-9a86-6ac522f742a3",
        "feedback_tokens": []
    }
}
```

最后，langserve中还提供了一个客户端，可以方便的调用langserve暴露出的服务。

```
from langserve import RemoteRunnable

client = RemoteRunnable("http://127.0.0.1:8000/langchainDemo")
client.invoke({"text":"nice to meet you", "language":"Chinese"})
```

2、使用Gradio快速构建前端页面

Gradio是Huggingface开源的一个用于构建交互式Web应用程序的库，他可以方便的构建一个前端页面，用来展示你的大模型服务。主打的也同样是一个简单。甚至不需要有HTML、CSS或者网页开发的经验，就可以快速构建一个用于验证大模型的前端页面。

首先同样是需要安装Gradio的依赖

```
#安装Gradio依赖。要求python版本>=3.10
!pip install -qU gradio
```

然后，对于Gradio，不需要太多的基础，直接来一个简单粗暴的案例，就能快速上手Gradio

```
import gradio as gr
def greet(name, intensity):
    return "Hello, " + name + "!" * int(intensity)

demo = gr.Interface(
    fn=greet,
    inputs=[gr.Text(label="Name"), gr.Slider(label="Intensity", minimum=0, maximum=10,
step=1)],
    outputs=[gr.Text(label="Output")],
)

demo.launch() #share=True表示可以基于HuggingFace的虚拟服务直接分享这个页面给其他人。国内一般无法正常
分享
```

```
# 停止服务
demo.close()
```

interface可以说是Gradio最核心的组件，直接指定了从inputs组件中输入数据，交由fn指定的函数进行处理，然后结果输出到outputs组件当中。Gradio中直接定义了非常多的组件，不需要管样式、布局，甚至不需要写HTML、CSS，直接声明你需要用哪些组件就行。Gradio所有支持的组件列表如下：

```
["Audio","BarPlot","Button","Chatbot","ChatMessage","ClearButton","Component","component""
get_component_instance","_Keywords","Checkbox","CheckboxGroup","Code","ColorPicker","Dataf
rame","DataFrame","Dataset","DownloadButton","DuplicateButton","Fallback","Form","FormComp
onent","Gallery","HTML","FileExplorer","Image","JSON","Json","Label","LinePlot","BrowserSt
ate","LoginButton","Markdown","MessageDict","Textbox","DateTime","Dropdown","Model3D","Fil
e","HighlightedText","AnnotatedImage","CheckboxGroup","Text","Highlightedtext","Annotatedi
mage","Highlight","Checkboxgroup","Number","Plot","Radio","ScatterPlot","Slider","State","
Timer","UploadButton","Video","StreamingInput","StreamingOutput","ImageEditor","ParamViewe
r","MultimodalTextbox","NativePlot","DeepLinkButton",
]
```

当然，声明组件的时候，也可以用交互式的方法声明。例如示例中声明的interface，等价于

```
demo = gr.Interface(
    fn=greet,
    inputs=[gr.Text(label="Name"), gr.Slider(label="Intensity", minimum=0, maximum=10,
step=1)],
    outputs=[gr.Text(label="Output")],
)
```

在interface中，还有很多补充的参数可以选择。例如 live=True，则会自动更新，只要输入框有变化，就会自动更新。

当然，除了这种抽盲盒式的构建页面，也可以像传统页面开发一样，自定义组件布局，定制组件的业务逻辑。例如：

```
import gradio as gr

with gr.Blocks() as demo:
    text_count = gr.State(1)
    add_btn = gr.Button("Add Box")
    add_btn.click(lambda x: x + 1, text_count, text_count)

    @gr.render(inputs=text_count)
    def render_count(count):
        boxes = []
        for i in range(count):
            box = gr.Textbox(key=i, label=f"Box {i}")
            boxes.append(box)

        def merge(*args):
            return " ".join(args)

        merge_btn.click(merge, boxes, output)

    merge_btn = gr.Button("Merge")
    output = gr.Textbox(label="Merged Output")

demo.launch()
```

```
demo.close()
```

有了Gradio后，我们就可以快速的复刻出之前langserve的playground页面了。 这里就不再重复了，直接给出代码。

```
import gradio as gr
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from config.load_key import load_key
from langchain_openai import ChatOpenAI

# 提示词模板
prompt_template = ChatPromptTemplate.from_messages([
    ("system","Translat the following from English into {language}"),
    ("user","{text}")
])
# 构建阿里云百炼大模型客户端
llm = ChatOpenAI(
    model="qwen-plus",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    openai_api_key=load_key("BAILIAN_API_KEY"),
)
```

```
# 结果解析器 StrOutputParser会AIMessage转换成为str，实际上就是获取AIMessage的content属性。
parser = StrOutputParser()
# 构建链
chain = prompt_template | llm | parser


def get_model_response(language, text):
    request = {"language":language, "text":text}
    response = chain.invoke(request)
    return response

def process_input(language, text):
    if language and text:
        response = get_model_response(language, text)
        return response
    else:
        return "Unsupported language"

with gr.Blocks() as demo:
    gr.Markdown("# LangServe Playground")
    with gr.Row():
        with gr.Column():
            gr.Markdown("## Try it")
            inputs_language = gr.Textbox(label="LANGUAGE*", placeholder="请输入语言",
value="中文")
            inputs_text = gr.Textbox(label="TEXT*", placeholder="请输入文本", value="how
are you")
            btn_start = gr.Button("Start", variant="primary")
        with gr.Column():
            output_text = gr.Textbox(label="Output")

    btn_start.click(process_input, inputs=[inputs_language, inputs_text], outputs=
[output_text])

demo.launch()
```

> 实际上，这个页面就是直接抓取截图，交给AI大模型生成的。如果你还有更多更复杂的页面交互设计，交给AI大模型是一个不错的选择。

总结

在课程中一直在强调，AI大模型是一个划时代的工具。但是如何把AI大模型这个工具用得更好，这是现在整个业界都在思考的问题。
这一章节是将AI大模型与传统的B/S应用进行整合，这样即能发挥AI大模型强大的推理能力，又能吸收传统B/S应用的业务经验。